SIMATS ENGINEERING
ASSESSMENT TOOL 6

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

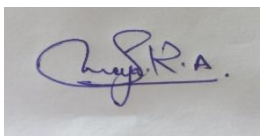
Annexure A

Dry Run Evaluation

Code of Conduct:

I Saranya.R.A(192572052) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:

Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

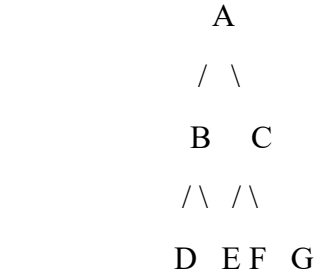
Iteration Queue Visited Node

1
2
3
4
5
6
7

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. **Initial State :**
   1 3 6
   5 0 2
   4 7 8

Goal State :
1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1			
2			
3			
4			
5			

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

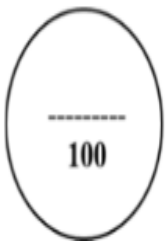
CO2 – ASSESSMENT TOOL 1

Application Based Questions

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

QUESTIONS: 3 **Duration:** 1 Hour **Total Marks:** 30

Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Total Marks (30)
Understanding of Problem Context	20% (2)	/2	/2	/2	/6
Application of Concepts	30% (3)	/3	/3	/3	/9
Problem-Solving Approach	20% (2)	/2	/2	/2	/6
Accuracy of Solution	20% (2)	/2	/2	/2	/6
Clarity	10% (1)	/1	/1	/1	/3
Total per Question	10 Marks	/10	/10	/10	/30



ANSWER

1. Breadth-First Search (BFS) Algorithm

Algorithm

Input: Graph , Start Node

Output: BFS Traversal Order

Step 1: Start.

Step 2: Create an empty Queue Q.

Step 3: Mark the start node S as visited.

Step 4: Enqueue S into Q.

Step 5: While Q is not empty, do:

a) Dequeue the front node N.

b) Visit (print) N.

c) For every adjacent node of N:

If the node is not visited:

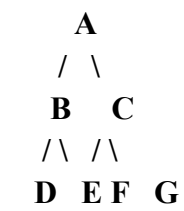
Mark it as visited.
Enqueue it into Q.

Step 6: Stop.

Explanation

- **BFS visits nodes level by level.**
- **It always visits all neighboring nodes before moving to the next level.**
- **It uses a Queue (FIFO – First In First Out) data structure.**

BFS Example



BFS Execution

Iteration	Queue Before	Visited	Queue After
1	A	A	B, C
2	B, C	B	C, D, E
3	C, D, E	C	D, E, F, G
4	D, E, F, G	D	E, F, G
5	E, F, G	E	F, G
6	F, G	F	G
7	G	G	Empty

BFS Traversal

A → B → C → D → E → F → G

2. Depth-First Search (DFS) Algorithm

Algorithm

Input: Graph , Start Node

Output: DFS Traversal Order

Step 1: Start.

Step 2: Create an empty Stack.

Step 3: Push the start node S into the stack.

Step 4: While stack is not empty:

a) Pop the top node N.

b) If N is not visited:

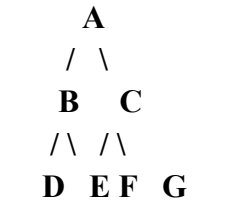
Visit N.
Mark N as visited.
Push all adjacent nodes of N
into the stack in reverse order.

Step 5: Stop.

Explanation

- **DFS explores one branch completely before moving to another branch.**
- **It uses a Stack (LIFO – Last In First Out).**

DFS Example



DFS Execution

Iteration	Stack Before	Visited	Stack After
1	A	A	C, B
2	C, B	B	C, E, D
3	C, E, D	D	C, E
4	C, E	E	C
5	C	C	G, F
6	G, F	F	G
7	G	G	Empty

DFS Traversal

A → B → D → E → C → F → G

3. Breadth-First Search Algorithm for 8-Puzzle

Algorithm

Input: Initial State, Goal State

Output: Shortest Path to Goal

Step 1: Start.

Step 2: Create an empty Queue.

Step 3: Insert the Initial State into the Queue.

Step 4: Mark Initial State as visited.

Step 5: Repeat until Queue becomes empty:

a) Remove the front state.

b) If it is Goal State:

Stop and print the solution.

c) Generate all possible child states by moving the blank tile (Up, Down, Left, Right).

d) If a child is not visited:

Mark it visited.

Add it to the Queue.

Step 6: If Queue becomes empty,

Goal State does not exist.

Step 7: Stop.

Explanation

- **BFS explores all states at one depth before moving to the next depth.**
- **Every move in the 8-puzzle has the same cost (one move).**
- **Therefore, the first time BFS reaches the goal state, it guarantees the minimum number of moves.**

Initial State

1 3 6
5 0 2
4 7 8
Goal State
1 2 3
4 5 6
7 8 0

First Expansion
The blank (0) is in the center, so it can move in 4 directions.
Generated states are:
Move Up
1 0 6
5 3 2
4 7 8
Move Down
1 3 6
5 7 2
4 0 8
Move Left
1 3 6
0 5 2
4 7 8
Move Right
1 3 6
5 2 0
4 7 8
These four states are inserted into the queue.

Why does BFS always find the shortest solution?
Because:

- **BFS searches level by level.**
- **Each move has an equal cost (1 move).**
- **The goal state is reached for the first time through the fewest possible moves.**
- **Hence, BFS always finds the optimal (shortest) solution for an unweighted 8-puzzle.**

Note: For the given initial state 1 3 6 / 5 0 2 / 4 7 8, the goal state is not reachable in only five iterations. A correct BFS trace would require many more expansions, so any worksheet showing only five iterations cannot legitimately reach the stated goal.